Write a C program to simulate page replacement algorithms.

a) FIFO
b) LRU
c) Optimal

Code:

```
#include <stdio.h>
#include <stdlib.h>

void fifo(int pages[], int n, int frames) {
    int frame[frames];
    int i, j, k = 0, page_faults = 0;
    for (i = 0; i < frames; i++) frame[i] = -1;

    printf("\n--- FIFO Page Replacement ---\n");
    for (i = 0; i < n; i++) {
        int found = 0;
        for (j = 0; j < frames; j++) {
            if (frame[j] == pages[i]) {
                found = 1;
                break;
            }
        }
        if (!found) {
            frame[k] = pages[i];
            k = (k + 1) % frames;
            page_faults++;
        }
        printf("Page %d -> [", pages[i]);
        for (j = 0; j < frames; j++) printf("%d ", frame[j]);
        printf("]\n");
    }
```

```
    printf("Total Page Faults (FIFO): %d\n", page_faults);
}

void lru(int pages[], int n, int frames) {
    int frame[frames], time[frames];
    int i, j, page_faults = 0, t = 0;
    for (i = 0; i < frames; i++) {
        frame[i] = -1;
        time[i] = 0;
    }

    printf("\n--- LRU Page Replacement ---\n");
    for (i = 0; i < n; i++) {
        int found = 0;
        for (j = 0; j < frames; j++) {
            if (frame[j] == pages[i]) {
                found = 1;
                t++;
                time[j] = t;
                break;
            }
        }
        if (!found) {
            int min_time = time[0], pos = 0;
            for (j = 1; j < frames; j++) {
                if (time[j] < min_time) {
                    min_time = time[j];
                    pos = j;
                }
            }
            frame[pos] = pages[i];
```

```
            t++;
            time[pos] = t;
            page_faults++;
        }
        printf("Page %d -> [", pages[i]);
        for (j = 0; j < frames; j++) printf("%d ", frame[j]);
        printf("]\n");
    }
    printf("Total Page Faults (LRU): %d\n", page_faults);
}

void optimal(int pages[], int n, int frames) {
    int frame[frames], i, j, page_faults = 0;
    for (i = 0; i < frames; i++) frame[i] = -1;

    printf("\n--- Optimal Page Replacement ---\n");
    for (i = 0; i < n; i++) {
        int found = 0;
        for (j = 0; j < frames; j++) {
            if (frame[j] == pages[i]) {
                found = 1;
                break;
            }
        }

        if (!found) {
            int replace_index = -1, farthest = i;
            for (j = 0; j < frames; j++) {
                int k;
                for (k = i + 1; k < n; k++) {
                    if (frame[j] == pages[k]) {
```

```
                if (k > farthest) {
                    farthest = k;
                    replace_index = j;
                }
                break;
            }
        }
        if (k == n) { // never used again
            replace_index = j;
            break;
        }
    }
    if (replace_index == -1) replace_index = 0;
    frame[replace_index] = pages[i];
    page_faults++;
  }
  printf("Page %d -> [", pages[i]);
  for (j = 0; j < frames; j++) printf("%d ", frame[j]);
  printf("]\n");
 }
 printf("Total Page Faults (Optimal): %d\n", page_faults);
}

int main() {
 int n, frames;
 printf("Enter number of pages: ");
 scanf("%d", &n);
 int pages[n];
 printf("Enter page reference string: ");
 for (int i = 0; i < n; i++) scanf("%d", &pages[i]);
 printf("Enter number of frames: ");
```

```
    scanf("%d", &frames);

    fifo(pages, n, frames);
    lru(pages, n, frames);
    optimal(pages, n, frames);

    return 0;
}
```

Output: